

SISTEMAS P2P, DHT Y BITTORRENT

1. Introducción

Un sistema distribuido es una colección de computadoras independientes que aparece como un sistema único. Uno de los desafíos centrales es cómo repartir los datos entre muchos nodos de forma que ninguno sea indispensable para el funcionamiento del sistema.

El modelo cliente-servidor resuelve esto de forma simple pero frágil: hay un servidor que tiene todo y clientes que piden lo que necesitan. Si el servidor falla, el sistema falla. Si el servidor se satura, todos esperan. Esta arquitectura contradice uno de los principios fundamentales de los sistemas distribuidos: evitar el punto único de fallo.

Las redes P2P son una respuesta concreta a ese problema. La idea es que los propios participantes almacenen y sirvan los datos. No hay un nodo especial que sea el servidor: cada nodo es simultáneamente cliente y servidor. Si un nodo falla, los demás siguen funcionando porque los datos están replicados entre ellos.

Esto conecta con los conceptos del apunte anterior: intuitivamente, un sistema P2P bien diseñado prioriza la disponibilidad ante fallos parciales por sobre la consistencia estricta. Sin embargo, el teorema CAP aplica formalmente a sistemas de almacenamiento distribuido con semántica de lectura/escritura consistente, y no debe usarse para clasificar formalmente a BitTorrent u otras redes P2P. La referencia al CAP se usa aquí solo como analogía para entender la prioridad de diseño.

2. Cliente-Servidor vs P2P

2.1 El modelo cliente-servidor

En el modelo cliente-servidor hay una asimetría estructural: el servidor posee los datos y los clientes los solicitan. La transferencia fluye siempre desde el servidor hacia los clientes.

Esta arquitectura es simple de implementar y suficiente para la mayoría de los casos cotidianos. Pero tiene dos limitaciones críticas cuando el número de clientes o el tamaño de los datos crece:

Cuello de botella en el servidor. El ancho de banda de subida del servidor es finito. Si N clientes piden el mismo archivo simultáneamente, ese ancho de banda se divide entre N .

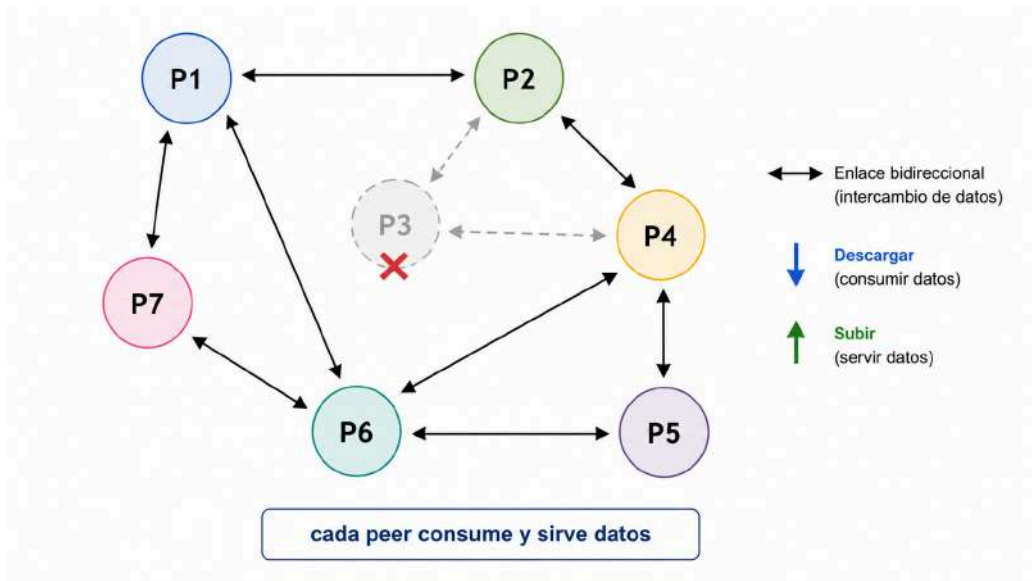
Punto único de fallo. Si el servidor cae, todos los clientes pierden acceso a los datos.



Modelo cliente-servidor: servidor central, cuello de botella y punto único de fallo

2.2 El modelo P2P

En el modelo P2P, cada nodo actúa simultáneamente como cliente y servidor. Cuando un peer descarga un archivo, también sirve las piezas que ya tiene a otros peers. La capacidad de distribución crece con cada nuevo participante, porque cada uno aporta ancho de banda de subida además de consumir ancho de banda de descarga.



Modelo P2P: cada nodo consume y sirve datos

La propiedad más importante: la escalabilidad es horizontal. Agregar más peers no sobrecarga el sistema sino que lo refuerza. A continuación un cuadro de comparación

Criterio	Cliente-Servidor	P2P
Quién sirve los datos	Solo el servidor	Todos los nodos con el archivo
Escalabilidad	Limitada por el servidor	Crece con el número de peers
Punto único de fallo	Sí (el servidor)	No (distribuido)
Consistencia de datos	Fácil de garantizar	Requiere diseño cuidadoso
Complejidad de implementación	Baja	Alta
Control centralizado	Sí	No (o parcial)
Ejemplos	HTTP, FTP, NFS	BitTorrent, DHT

3. P2P no estructurado y flooding

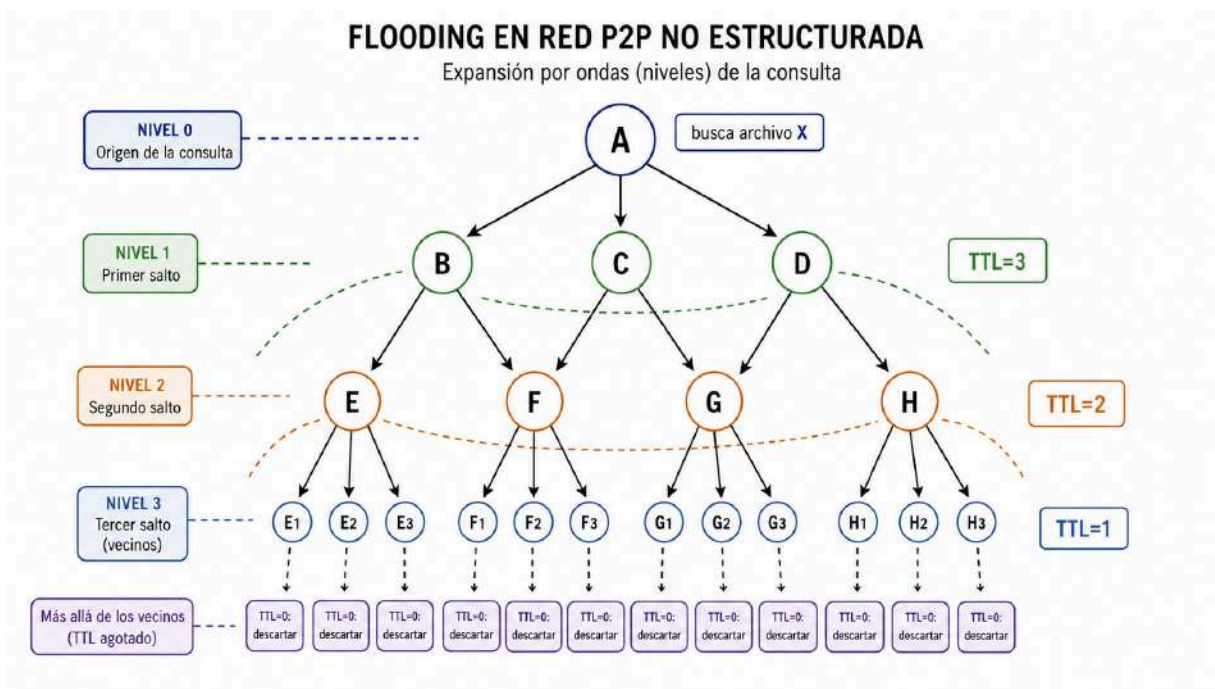
3.1 Qué es una red P2P no estructurada

En una red P2P no estructurada los nodos se conectan entre sí de forma aleatoria. No hay reglas sobre qué peer se conecta con cuál ni sobre cómo se distribuyen los datos entre nodos. La topología resultante es irregular y cambia constantemente a medida que los nodos entran y salen —fenómeno llamado churn. Esta aleatoriedad tiene una ventaja: el sistema es robusto ante el churn. Si un nodo desaparece, no hay ninguna estructura lógica que reconstruir. Pero introduce un problema fundamental: si no hay organización, ¿cómo encuentra un nodo el archivo que busca?

3.2 Búsqueda por flooding

La respuesta es el flooding (inundación): el nodo que quiere un archivo envía la consulta a todos sus vecinos directos. Cada vecino que no tiene el archivo lo reenvía a los suyos. El proceso se repite hasta que algún nodo responde con el archivo, o hasta que se alcanza el límite de saltos.

Ese límite es el TTL (Time To Live). Cada vez que la consulta avanza un salto, el TTL se decrementa en uno. Al llegar a cero, el mensaje se descarta y no se propaga más.

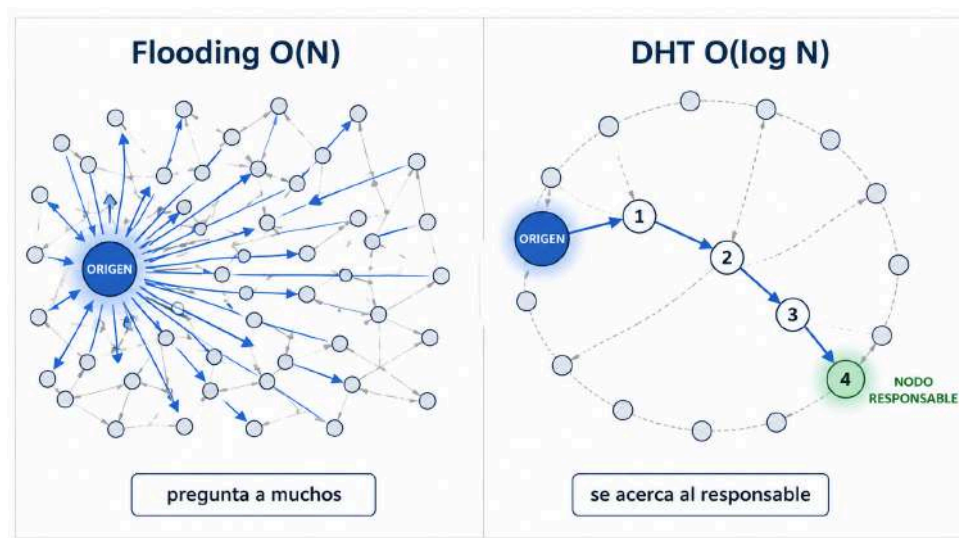


Búsqueda por flooding con TTL = 3

3.3 El problema de escalabilidad del flooding

El flooding consume ancho de banda de forma proporcional al número de nodos contactados. En el peor caso, sin TTL, se contactan todos los N nodos: complejidad $O(N)$.

El TTL mitiga el problema pero introduce otro: si es demasiado bajo, la búsqueda falla aunque el archivo exista a más saltos. El P2P no estructurado con flooding es simple y robusto ante churn, pero no sirve para redes de millones de nodos. Se necesita una alternativa que localice recursos de forma eficiente sin contactar a todos. Esa alternativa es el P2P estructurado con DHT.



Flooding pregunta a muchos; DHT enruta hacia el responsable

4. P2P estructurado y DHT

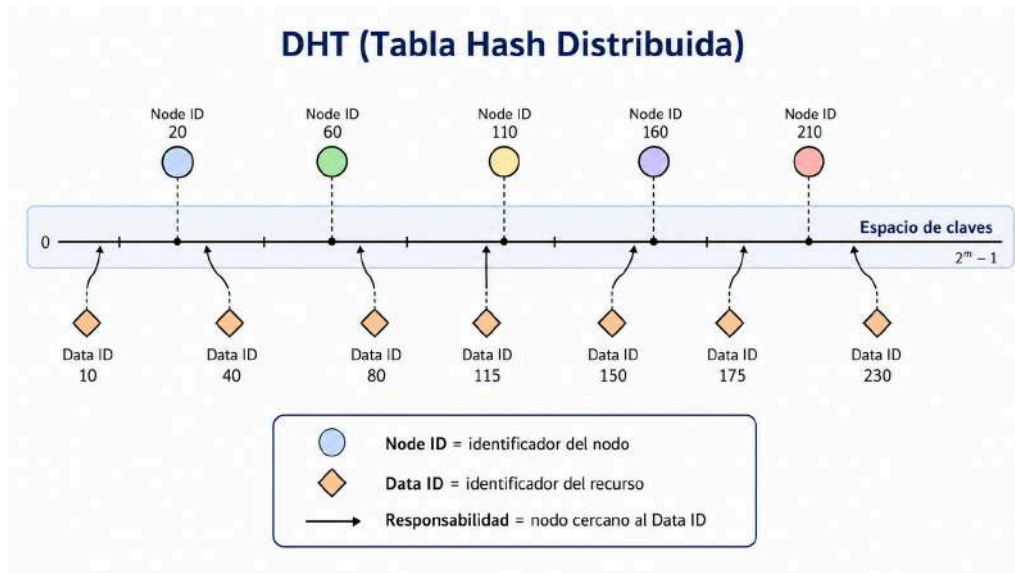
4.1 La idea del P2P estructurado

En una red P2P estructurada, los nodos se organizan siguiendo una topología lógica definida —típicamente un anillo o árbol— que permite localizar cualquier recurso en $O(\log N)$ saltos en condiciones normales y con tablas de ruteo mantenidas correctamente.

La topología es lógica: no depende de la ubicación física ni de la latencia real de la red. Se construye sobre los identificadores numéricos de los nodos. El mecanismo que hace posible esta organización es la Tabla Hash Distribuida (DHT).

4.2 Qué es una DHT

Una DHT es una estructura de datos distribuida que implementa una tabla clave-valor repartida entre muchos nodos. Cada nodo es responsable de almacenar y responder consultas sobre un subconjunto de las claves. El conjunto de todos los nodos cubre el espacio completo. A continuación presentamos los tres conceptos centrales:



DHT: Node ID, Data ID y regla de responsabilidad

Espacio de claves. Un rango numérico finito, típicamente $[0, 2^{160} - 1]$ con SHA-1. Todos los identificadores, de nodos y de recursos, viven en este mismo espacio numérico.

Node ID. Identificador único asignado a cada nodo al unirse a la red DHT, generado como hash de IP:puerto o aleatoriamente.

Data ID. Identificador de un recurso, generado aplicando una función hash a su contenido o nombre. También vive en el mismo espacio de claves.

La propiedad fundamental: dado un Data ID cualquiera, la DHT puede determinar qué nodo es responsable de ese recurso y enrutar una consulta hasta él en $O(\log N)$ saltos (en condiciones normales y con tablas de ruteo actualizadas).

Concepto	Definición	Cómo se genera
Node ID	Identificador único del nodo en la DHT	Hash de IP:puerto, o aleatorio
Data ID	Identificador del recurso	Hash del contenido o nombre del recurso
Espacio de claves	Rango numérico compartido por Node IDs y Data IDs	$[0, 2^{160} - 1]$ con SHA-1
Responsabilidad	Qué nodo guarda información sobre un recurso	El nodo cuyo ID es el más cercano al Data ID

4.3 Por qué DHT es superior al flooding

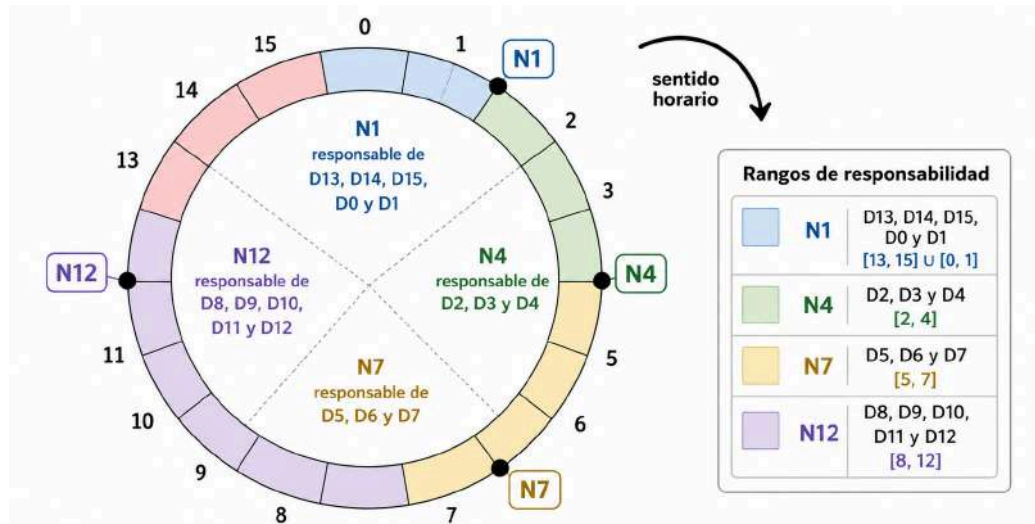
El flooding no sabe dónde está el recurso y pregunta a todo el mundo. La DHT calcula hacia dónde dirigirse en cada paso, acercándose al responsable en forma logarítmica.

N (nodos en la red)	Flooding $O(N)$ — peor caso	DHT $O(\log N)$ — condiciones normales
1.000	1.000 mensajes	~10 mensajes
1.000.000	1.000.000 mensajes	~20 mensajes
1.000.000.000	1.000.000.000 mensajes	~30 mensajes

5. Chord

5.1 El anillo lógico

Chord organiza los nodos en un anillo lógico ordenado por sus Node IDs. Si el espacio es $[0, M-1]$, los nodos se distribuyen en ese anillo en orden ascendente, y el nodo con ID más alto se conecta de vuelta con el de ID más bajo, cerrando el anillo.



Chord: asignación de claves por regla del sucesor

5.2 La regla del sucesor

Regla fundamental: cada Data ID es responsabilidad del primer nodo activo que se encuentra avanzando en sentido horario desde el valor de esa clave. A ese nodo se le llama el sucesor de la clave, y se escribe $\text{successor}(k)$.

Caso borde: si una clave es mayor que todos los Node IDs del anillo, da la vuelta y el responsable es el nodo con el ID más bajo.

Ejemplo: Asignación básica de claves. Anillo: espacio $[0, 15]$, nodos activos: N1, N4, N7, N12.

Clave	Razonamiento	Nodo responsable
D0	Avanzando desde 0: el primer nodo activo es N1 (ID=1)	N1
D1	ID=1 es exactamente N1	N1
D2	Desde 2 en sentido horario: N4 (ID=4) es el primero	N4
D3	Desde 3: N4 (ID=4) es el primero	N4
D5	Desde 5: N7 (ID=7) es el primero	N7
D8	Desde 8: N12 (ID=12) es el primero	N12
D13	Desde 13: pasamos 13, 14, 15 sin nodos activos, damos vuelta, llegamos a N1	N1
D15	Desde 15: pasamos 15, damos vuelta, llegamos a N1	N1

Rangos de responsabilidad completos:

- N1: D0, D1 (directos) y D13, D14, D15 (los que dan vuelta)
- N4: D2, D3, D4
- N7: D5, D6, D7
- N12: D8, D9, D10, D11, D12

Ejemplo Entrada y salida de nodos

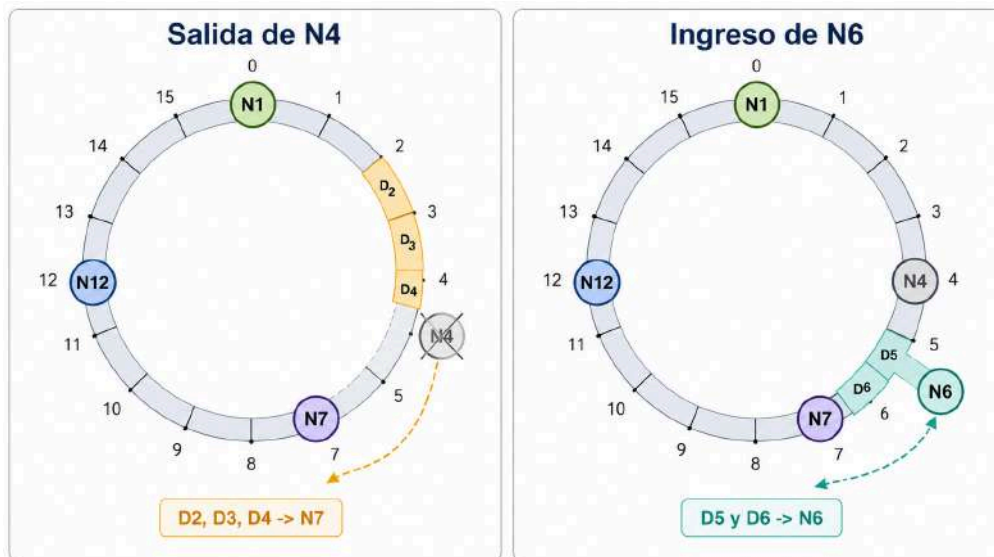
Anillo: espacio [0, 15], nodos activos: N1, N4, N7, N12.

Caso A — N4 abandona la red: las claves D2, D3, D4 quedan sin responsable. La regla del sucesor dice que pasan al primer nodo activo en sentido horario desde esas posiciones. Con N4 ausente, ese nodo es N7.

- N7 asume: D2, D3, D4, D5, D6, D7
- N1 y N12 no se ven afectados

Caso B — Se incorpora N6: N6 se inserta entre N4 y N7. Las claves D5 y D6 eran responsabilidad de N7. Ahora N6 es el primer nodo activo desde esas posiciones.

- N6 asume: D5, D6
- N7 retiene: D7
- N4 no se ve afectado



Chord ante churn: salida de N4 e ingreso de N6

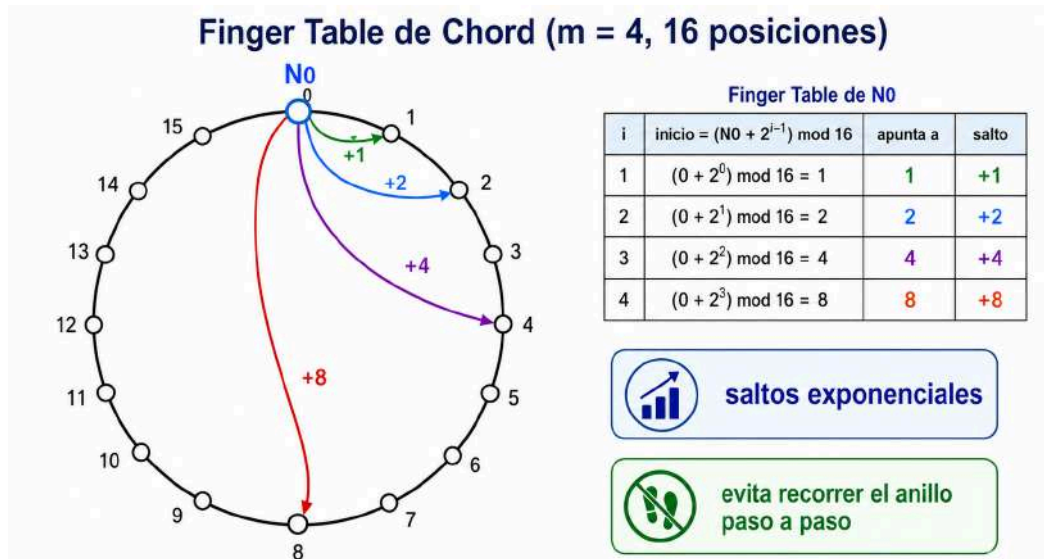
5.5 La Finger Table — búsqueda $O(\log N)$

Si cada nodo conoce solo a su sucesor inmediato, localizar el responsable de una clave podría requerir $O(N)$ saltos. Chord resuelve esto con la Finger Table.

Cada nodo mantiene una tabla de K entradas. La entrada i del nodo con ID n apunta al sucesor de la clave $(n + 2^i)$ módulo 2^m , donde m es el tamaño del espacio de claves en bits. Esto crea saltos exponenciales: en

cada paso, el nodo salta a la posición más lejana posible sin pasarse del destino. Resultado: cualquier clave se localiza en $O(\log N)$ saltos (con la tabla de ruteo mantenida correctamente).

Ejemplo conceptual: un nodo N_0 en un anillo de 16 posiciones ($m=4$) tendría entradas apuntando hacia N_0+1 , N_0+2 , N_0+4 , N_0+8 . En lugar de recorrer el anillo paso a paso, salta directamente a posiciones cada vez más lejanas.



Finger Table: saltos exponenciales para buscar en $O(\log N)$

6. Kademlia

6.1 Idea general

La DHT Mainline de BitTorrent está basada en Kademlia. Comparte con Chord la idea fundamental de organizar nodos en un espacio de claves con enrutamiento eficiente, pero difiere en la métrica de distancia y en la estructura de enrutamiento.

En Chord, la distancia es numérica en un anillo. En Kademlia, la distancia es la operación XOR bit a bit. Esta diferencia produce propiedades distintas: en particular, Kademlia tiene mejor comportamiento ante el churn porque las tablas de ruteo se actualizan de forma más simétrica.

6.2 La métrica XOR

Definición: la distancia lógica entre dos IDs A y B es $\text{distance}(A, B) = A \text{ XOR } B$.

El operador XOR produce un 1 en cada bit donde A y B difieren, y un 0 donde coinciden. El resultado es un número que indica cuán diferentes son los dos IDs bit a bit.

Propiedades:

- Simétrica: $\text{distance}(A, B) = \text{distance}(B, A)$.
- Identidad: $\text{distance}(A, A) = 0$.
- Cercanía por prefijo: dos IDs que comparten un prefijo binario largo tienen distancia XOR pequeña. El bit más significativo del resultado XOR determina el nivel de separación en el árbol binario (trie) de Kademlia.

Por qué el prefijo importa: en Kademlia, los nodos se organizan en un árbol binario donde la posición depende del ID binario. Dos nodos en la misma rama comparten un prefijo largo y son cercanos en la métrica XOR. El enrutamiento selecciona en cada paso el nodo conocido cuyo ID comparte el prefijo más largo con el destino.

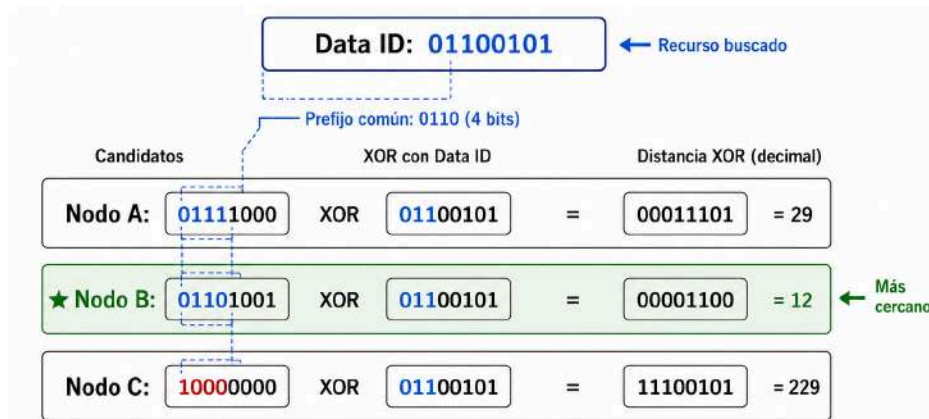
Ejemplo de Cálculo XOR básico

Data ID: **01100101**

← Recurso buscado

Nodos candidatos:

- Nodo A: 01111000
- Nodo B: 01101001
- Nodo C: 10000000



Paso 2 — Comparar distancias:

Nodo	Resultado XOR	Decimal
A	00011101	29
B	00001100	12 <-- menor
C	11100101	229

Nodo B es el más cercano (distancia XOR = 12).

Paso 3 — Verificar por prefijo:

- Recurso 01100101 y Nodo B 01101001: prefijo común 0110 (4 bits antes de divergir).
- Recurso 01100101 y Nodo A 01111000: prefijo común 011 (3 bits).
- Recurso 01100101 y Nodo C 10000000: ningún bit en común (MSB diferente).

El nodo con mayor prefijo común es el más cercano en XOR. Esto confirma que B es el mejor candidato

Ejemplo de salto de enrutamiento

Situación: el nodo actual tiene ID 00110000. Quiere enrutar una consulta hacia el recurso con Data ID 01101010. Conoce tres vecinos:

- Vecino V1: 00111100
- Vecino V2: 01100011
- Vecino V3: 11001010

Objetivo: elegir el vecino cuya distancia XOR al recurso sea la menor.

Recurso: 0 1 1 0 1 0 1 0

Vecino V1: 0 0 1 1 1 1 0 0

XOR: 0 1 0 1 0 1 1 0 = 64+16+4+2 = 86 decimal

Recurso: 0 1 1 0 1 0 1 0
 Vecino V2: 0 1 1 0 0 0 1 1
 XOR: 0 0 0 0 1 0 0 1 = 8+1 = 9 decimal

Recurso: 0 1 1 0 1 0 1 0
 Vecino V3: 1 1 0 0 1 0 1 0
 XOR: 1 0 1 0 0 0 0 0 = 128+32 = 160 decimal

Decisión: enrutar hacia V2 (distancia XOR = 9). V2 comparte el prefijo 0110 (4 bits) con el recurso, el mayor de los tres vecinos. En el siguiente salto, V2 aplicará el mismo criterio con sus propios vecinos, acercando la consulta progresivamente al nodo más cercano al Data ID.

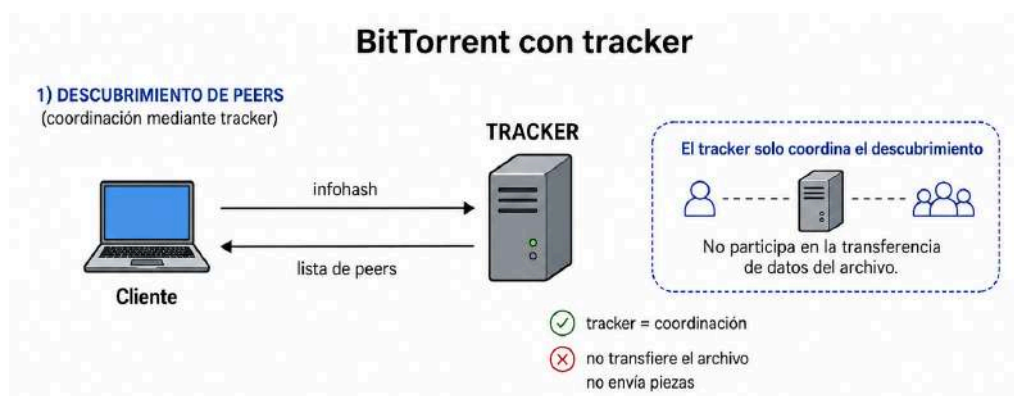
7. BitTorrent con tracker

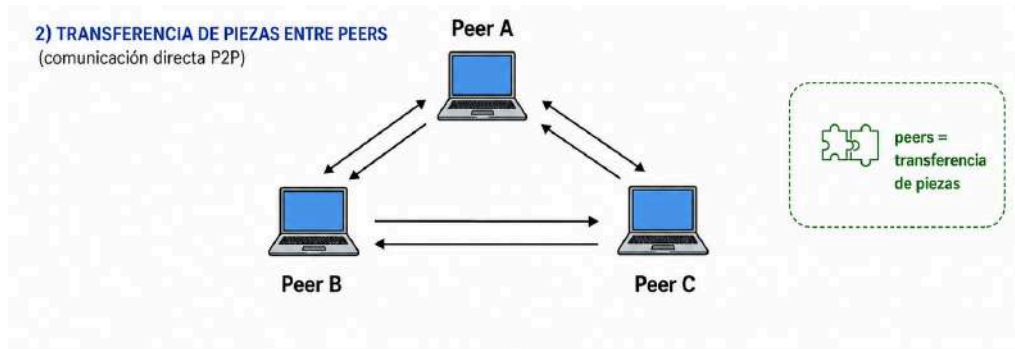
7.1 Terminología

Término	Definición
Torrent	Archivo de metadatos (.torrent) que describe el archivo: tamaño total, hashes de cada pieza, URL del tracker.
Infohash	Hash SHA-1 del archivo torrent. Identificador único del torrent en la red. Funciona como Data ID en la DHT de BitTorrent.
Peer	Cualquier nodo participando en la descarga o distribución de un torrent.
Seeder	Peer con la copia completa del archivo. Contribuye exclusivamente con upload.
Leecher	Peer descargando activamente. Tiene partes del archivo y las comparte mientras descarga.
Swarm	Conjunto de todos los peers participando en un mismo torrent (seeders + leechers).
Tracker	Servidor que mantiene el registro de qué peers están activos en cada swarm. No almacena ni transfiere el archivo.
Pieza (piece)	Fragmento del archivo de tamaño fijo (256 KB a 1 MB). Unidad de intercambio entre peers.
Magnet link	Enlace que contiene solo el infohash (y metadatos mínimos), sin URL de tracker. Requiere DHT para funcionar.

7.2 El modelo híbrido

BitTorrent con tracker es un modelo híbrido: usa un servidor central (el tracker) para que los peers se encuentren entre sí, pero la transferencia del archivo es completamente descentralizada entre peers. El tracker no almacena ni sirve el archivo: solo mantiene el registro de participantes activos en cada swarm.





BitTorrent con tracker: coordinación central, transferencia P2P

7.3 Flujo completo de descarga con tracker

1. El usuario descarga el archivo .torrent desde un sitio web.
2. El cliente BitTorrent lee el .torrent y extrae: el infohash, la URL del tracker, el tamaño total y los hashes de cada pieza.
3. El cliente contacta al tracker enviando: su dirección IP, su puerto, el infohash y su estado actual
4. El tracker devuelve una lista de peers activos en el swarm de ese infohash.
5. El cliente se conecta directamente a esos peers e intercambia piezas
6. Periódicamente, el cliente actualiza su lista de peers consultando al tracker.
7. Al completar la descarga, el cliente se convierte en seeder y puede continuar subiendo piezas.

7.4 Limitaciones del tracker centralizado

Punto único de fallo: si el tracker cae, los peers ya conectados pueden seguir intercambiando piezas (ya conocen a sus vecinos), pero ningún peer nuevo puede unirse al swarm.

Censura: bloquear el tracker impide que nuevos usuarios accedan al swarm.

Privacidad: el tracker puede registrar qué IP:puerto participa en un swarm asociado a un infohash.

8. BitTorrent trackerless con DHT

8.1 La motivación

La solución a los problemas del tracker es distribuir su función entre todos los peers. Cada peer actúa como un mini-registro: almacena y responde consultas sobre qué peers tienen determinados torrents.

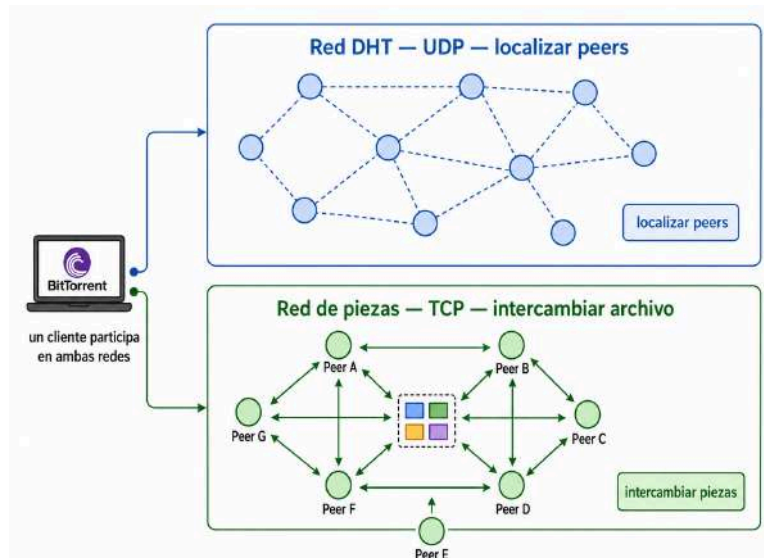
Para que esto funcione sin preguntar a todos, se necesita un mecanismo de localización eficiente. La DHT Mainline de BitTorrent está basada en Kademlia y usa la métrica XOR para enrutar consultas de forma eficiente sin servidor central.

8.2 El infohash como Data ID

La integración entre BitTorrent y Kademlia se apoya en que el infohash funciona como un Data ID en la DHT. Cuando un peer quiere descargar un torrent con infohash H, consulta a la DHT: ¿qué peers tienen piezas del torrent identificado por H?

La consulta se enruta progresivamente hacia los nodos cuyo Node ID es más cercano a H según la métrica XOR. En cada salto, el nodo contactado responde con los nodos de su tabla de ruteo que estén más cerca de H que él mismo. El proceso converge en $O(\log N)$ saltos (en condiciones normales) hacia los nodos más cercanos al infohash.

Esos nodos más cercanos almacenan la lista de peers que han anunciado tener ese torrent. A diferencia de Chord, no hay un único nodo sucesor rígido: en Kademlia, múltiples nodos cercanos al Data ID pueden colaborar en almacenar y responder la información, lo que mejora la robustez ante el churn.



BitTorrent trackerless usa dos redes: DHT para encontrar peers y TCP para piezas

8.3 Flujo completo de descarga trackerless

1. El usuario obtiene un magnet link con el infohash (sin URL de tracker, sin archivo .torrent).
2. El cliente se conecta a nodos conocidos de entrada a la red DHT (llamados nodos de bootstrap): son direcciones de nodos estables incluidas en el cliente. Estos nodos no coordinan el swarm ni almacenan el contenido; solo permiten al nuevo cliente unirse a la red DHT.
3. Una vez en la DHT, el cliente usa el infohash como Data ID y enruta la consulta hacia los nodos con Node ID más cercano al infohash, según la métrica XOR.
4. En $O(\log N)$ saltos (en condiciones normales), la consulta llega a los nodos más cercanos al infohash, que devuelven la lista de peers que anunciaron estar en ese swarm.
5. El cliente se conecta a esos peers e intercambia piezas normalmente.
6. El cliente anuncia su propia presencia en la DHT para que otros que busquen el mismo torrent lo encuentren

8.4 Distinción importante: peer vs nodo en BitTorrent DHT

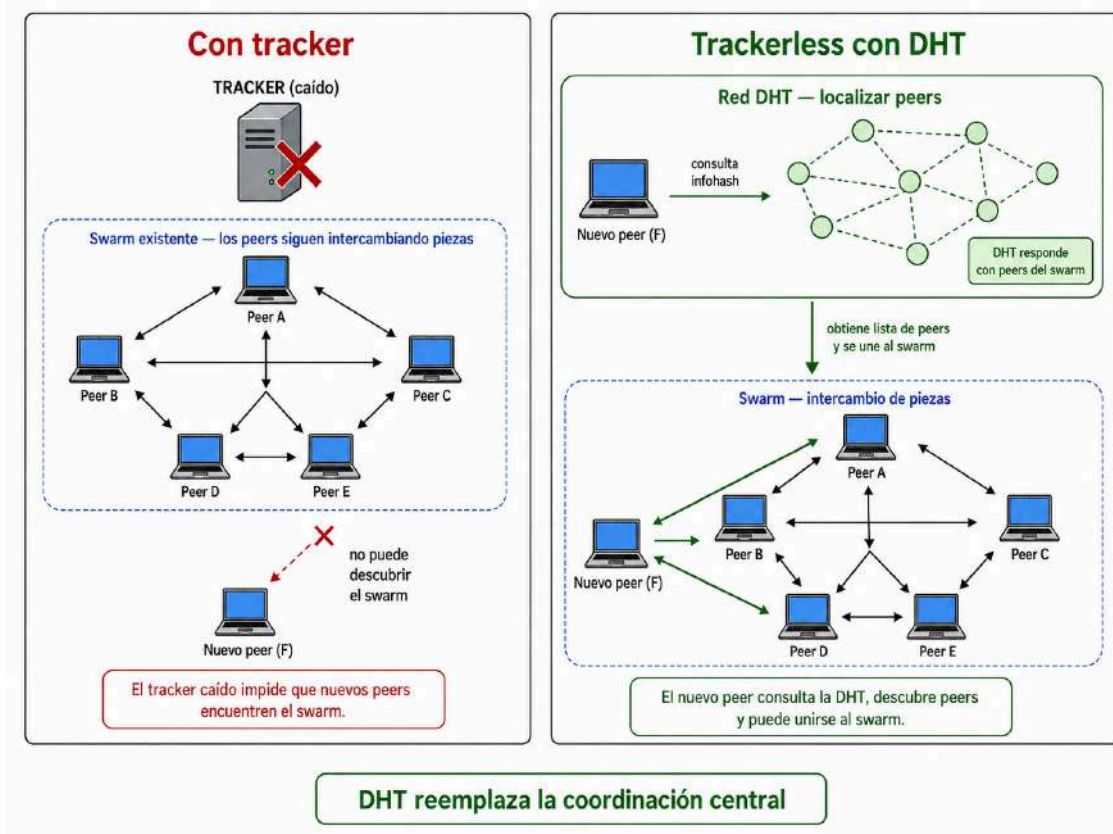
En BitTorrent trackerless coexisten dos redes paralelas:

- Red de piezas (TCP): los peers intercambian fragmentos del archivo usando el protocolo BitTorrent.
- Red DHT (UDP): los nodos participan en la DHT Kademlia para localizar peers que tienen un torrent.

En la práctica, el cliente BitTorrent moderno participa en ambas redes simultáneamente.

9. Comparación tracker vs trackerless

Criterio	Con tracker	Trackerless (DHT)
Coordinación de peers	Servidor central (tracker)	Distribuida en la DHT
Punto único de fallo	Sí (el tracker)	No
Nuevos peers si el tracker cae	No pueden unirse	Pueden unirse siempre
Censura	Bloquear el tracker bloquea el swarm	Muy difícil
Privacidad	El tracker puede registrar qué IP:puerto participa en cada swarm	Sin servidor central que registre
Inicio de descarga	Archivo .torrent con URL del tracker	Magnet link con infohash
Bootstrap necesario	Ninguno (URL en el .torrent)	Nodos conocidos de entrada a la DHT
Peers existentes si tracker cae	Siguen intercambiando entre sí	Sin efecto (no hay tracker)



Si cae el tracker: diferencia entre BitTorrent con tracker y trackerless

Caso comparativo detallado

Suponga que el tracker de un torrent popular cae repentinamente. Hay 500 seeders y 2.000 leechers activos.

Con tracker: los 2.500 peers ya conectados pueden seguir intercambiando piezas porque ya conocen a sus vecinos. Sin embargo, ningún nuevo usuario puede unirse al swarm: sin el tracker, no puede descubrir qué peers existen. El swarm irá reduciéndose a medida que los peers terminen y se desconecten, sin que nuevos puedan reemplazarlos.

Con DHT: la caída del tracker es irrelevante porque no existe. Cualquier nuevo usuario localiza el swarm a través de la DHT usando el infohash. El swarm puede crecer o mantenerse con normalidad.

Referencias

- Tanenbaum, A. S., & Bos, H. (2023). Modern Operating Systems (5th ed.). Pearson. Capítulo 11.
- Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson. Capítulo 2.
- Stoica et al. (2001). Chord: A scalable peer-to-peer lookup service for internet applications. ACM SIGCOMM.
- Maymounkov & Mazières (2002). Kademlia: A peer-to-peer information system based on the XOR metric. IPTPS.
- Cohen, B. (2003). Incentives build robustness in BitTorrent. Workshop on Economics of P2P Systems.